

Module 12 — Productionisation

From prototype to production-ready AI

🎯 Learning goals

- Implement **structured tracing** with trace IDs and spans for every request.
- Apply **reliability patterns**: retries with backoff, timeouts, circuit breakers, fallbacks.
- Enforce **cost controls**: token budgets, model tiering, caching.
- Deploy with **environment config**, secrets management, containers, and health checks.



Structured tracing

Know exactly what happened on every request

Why tracing matters

- A crew member reports a wrong answer this morning. Which request was it, and what did the model see?
- Every request gets a **trace ID** — one identifier carried through every step it touches.
- Each step is a **span** with timing, status, and metadata.
- Log spans as JSON. In production, send to **Datadog**, **Jaeger**, or **OpenTelemetry**.

TraceContext

```
class TraceContext:
    def __init__(self):
        self.trace_id = str(uuid.uuid4())
        self.spans: list[dict] = []

    def start_span(self, name: str) -> dict:
        span = {
            "span_id": str(uuid.uuid4()),
            "trace_id": self.trace_id,
            "name": name,
            "start_time": time.time(),
        }
        self.spans.append(span)
        return span

    def end_span(self, span, status="ok", metadata=None):
        span["end_time"] = time.time()
        span["duration_ms"] = (span["end_time"] - span["start_time"]) * 1000
        span["status"] = status
        if metadata:
            span["metadata"] = metadata
```

- → Every request gets a **trace_id** — propagate it through every function.
- → Each span captures **timing** and **status** — find the slow step instantly.
- → Log as **JSON** — structured logs are searchable, unstructured logs are not.



Reliability patterns

Dependencies fail. Decide what happens next.

What can go wrong

- **Network blips** — transient connection failures to LLM APIs.
- **Rate limits** — API returns 429 when you hit the quota.
- **Timeouts** — the model takes too long and your server thread is still waiting.
- **Cascading failures** — one slow dependency ties up the resources every other request needs.
- Four answers: **retries**, **timeouts**, **circuit breakers**, **fallbacks**.

Retries with exponential backoff

```
async def retry_with_backoff(fn, max_retries=3, base_delay=1.0):
    for attempt in range(max_retries):
        try:
            return await fn()
        except TransientError:
            if attempt == max_retries - 1:
                raise
            delay = base_delay * (2 ** attempt) + random.uniform(0, 0.5)
            await asyncio.sleep(delay)
```

- → **Exponential backoff**: 1s → 2s → 4s — gives the service time to recover.
- → **Jitter** (random.uniform) spreads out clients that all failed at the same moment.
- → Only retry on **transient errors** (5xx, timeout, rate limit) — never on 4xx.

Circuit breaker

```
class CircuitBreaker:
    def __init__(self, failure_threshold=5, recovery_time=60):
        self.failures = 0
        self.threshold = failure_threshold
        self.recovery_time = recovery_time
        self.state = "closed"  # closed = normal, open = failing

    async def call(self, fn):
        if self.state == "open":
            if time.time() - self.last_failure > self.recovery_time:
                self.state = "half-open"  # allow one test request
            else:
                raise CircuitOpenError("Service unavailable")
        try:
            result = await fn()
            self.failures = 0
            self.state = "closed"
            return result
        except Exception:
            self.failures += 1
            self.last_failure = time.time()
            if self.failures >= self.threshold:
                self.state = "open"
            raise
```

- → **Closed** = normal operation. **Open** = requests rejected locally, none sent.
- → After recovery_time, enters **half-open** — one test request decides.
- → A dead dependency stops occupying connections that healthy requests need.

Without vs with reliability patterns

No protection

No timeout → the thread waits on a call that never returns

Rate limit → immediate retry → rate limited again

Service down → every request waits its full timeout

The crew sees errors and spinners

With patterns

Timeout fires → fallback response in 2 seconds

Backoff + jitter → recovers without a coordinated retry spike

Circuit breaker → reject locally, try the fallback model

The crew gets an answer, sometimes a degraded one

🔄 Fallback chains

- When the primary model fails, fall through the chain rather than returning an error.
- Fallback chain: **primary model** → **cheaper model** → **cached response** → **plain apology**.
- Each rung is a worse answer than the one above it, and all of them beat a 500.
- `asyncio.wait_for(call, timeout=30)` — every external call gets a timeout.



Cost controls

An agent loop has no natural stopping point

Token budget tracking

```
class CostTracker:
    def __init__(self, session_budget=10_000, daily_budget=1_000_000):
        self.session_usage = 0
        self.daily_usage = 0
        self.session_budget = session_budget
        self.daily_budget = daily_budget

    def record(self, prompt_tokens: int, completion_tokens: int):
        total = prompt_tokens + completion_tokens
        self.session_usage += total
        self.daily_usage += total

    def within_budget(self) -> bool:
        return (self.session_usage < self.session_budget
                and self.daily_usage < self.daily_budget)
```

- → Track usage **per session** and **per day**. Reject requests over budget.
- → **Model tiering** is the biggest lever: classification → cheap model, reasoning → expensive.
- → **Caching** similar queries avoids redundant LLM calls entirely.



Deployment

Config, secrets, containers, health checks

Deployment essentials

- **Environment config** — `python-dotenv` for dev, real env vars in production. Never hard-code.
- **Secrets** — API keys in `.env` (gitignored), secrets manager in production.
- **Containers** — Dockerfile packages the app with its dependencies for consistent deployment.
- **Health checks** — `/health` endpoint returns 200 when the service is ready. Check dependencies.
- Provider-specific mappings: **AWS / Azure / GCP** sections below.

Dockerfile for an AI API

```
FROM python:3.12-slim
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=5s \
    CMD python -c "import httpx; httpx.get('http://localhost:8000/api/health').raise_for_s

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

- → **HEALTHCHECK** — orchestrators use this to route traffic only to healthy instances.
- → **Slim base image** — smaller attack surface, faster builds.
- → Copy requirements first — Docker caches the layer, rebuilds only when deps change.



Section — Cloud production

Map patterns to your provider

Production patterns at a glance

AWS

Run API: App Runner, ECS Fargate
Secrets: Secrets Manager, SSM
Logs/trace: CloudWatch, X-Ray, ADOT
Managed LLM: Amazon Bedrock
Cost: AWS Budgets, quotas

Azure

Run API: Container Apps, App Service
Secrets: Key Vault
Logs/trace: App Insights, Monitor
Managed LLM: Azure OpenAI
Cost: Cost Management, AOAI quotas

GCP

Run API: Cloud Run, GKE
Secrets: Secret Manager
Logs/trace: Cloud Logging, Trace
Managed LLM: Vertex AI
Cost: Billing budgets, quotas



Section — AWS

Enterprise AI on the most common cloud



When teams choose AWS

AWS is used when an organisation already runs on Amazon infrastructure or needs broad service coverage and compliance programs. For AI production you keep the same patterns from this module — trace IDs, retries, budgets — but wire them to **CloudWatch**, **Secrets Manager**, and optionally **Bedrock** instead of calling OpenAI directly. You deploy your FastAPI image to **App Runner** or **Fargate** and inject secrets at runtime, not in the Docker image.

AWS service mapping

- **Run container** — App Runner (simplest) or ECS Fargate for a FastAPI image.
- **Secrets** — Secrets Manager or SSM Parameter Store for `OPENAI_API_KEY`; never bake into the image.
- **Tracing** — JSON logs with `trace_id` → CloudWatch Logs; optional X-Ray or ADOT OpenTelemetry.
- **Managed LLM** — Amazon Bedrock when policy requires models inside your AWS account.
- **Cost** — AWS Budgets + service quotas; align with in-app `CostTracker` session/daily limits.



Section — Azure

Microsoft cloud + Azure OpenAI

When teams choose Azure

Azure fits Microsoft-centric enterprises: Entra ID, existing subscriptions, and **Azure OpenAI Service** for GPT models behind a private endpoint. Production AI apps use **Container Apps** or **App Service** for the FastAPI container, **Key Vault** for API keys, and **Application Insights** for the same span JSON you log in `TraceContext`. The course code stays portable Python; on Azure you swap the OpenAI client endpoint and deployment name while keeping tracing and cost patterns unchanged.

Azure service mapping

- **Run container** — Azure Container Apps (scale-to-zero) or App Service with Linux containers.
- **Secrets** — Key Vault references injected as env vars into Container Apps / App Service.
- **Tracing** — Application Insights ingests structured logs and dependency spans.
- **Managed LLM** — Azure OpenAI: resource endpoint + deployment name (not raw api.openai.com).
- **Cost** — Cost Management budgets; AOAI TPM/RPM quotas mirror token budget enforcement.



Section — GCP

Cloud Run + Vertex AI



When teams choose GCP

GCP is strong for container-native AI APIs: **Cloud Run** runs a single Docker image with minimal ops, and **Vertex AI** hosts Gemini and other models in-region. Teams on Google Cloud map `TraceContext` JSON to **Cloud Logging**, store secrets in **Secret Manager**, and use **Cloud Trace** for latency breakdowns. The module's Dockerfile maps cleanly to Cloud Run — one service, env-based config, `/health` for readiness probes.

GCP service mapping

- **Run container** — Cloud Run (best fit for one FastAPI container) or GKE for larger fleets.
- **Secrets** — Secret Manager mounted as env vars or volumes at deploy time.
- **Tracing** — Cloud Logging + Cloud Trace; OpenTelemetry export supported.
- **Managed LLM** — Vertex AI (Gemini, Model Garden) when data residency requires Google-hosted models.
- **Cost** — Billing budgets and quotas; pair with app-level `CostTracker` for per-session caps.

Summary

Trace every request

A trace ID is cheap. Debugging without one is expensive.

Retry with backoff, then fall back

Jitter stops a thundering herd. A cheaper model or a cache is still an answer.

Budget tokens before you spend them

Track per-session and per-day. Check the cap before the call, not after the bill.

Config and secrets stay out of the code

Environment variables or a secrets manager. A health check for the container.



Module 12 — Complete

Next: Module 13 — Edge Topics